CSE232 시스템프로그래밍(캡스톤디자인)

7주차 1차시 · V03

프로세스 기본: 주소 공간·상태·자원

경민기

KOREATECH
한국기술교육대학교

오늘의 지도

1부 주소 공간	2부 상태	3부 자원	4부 정리
코드 · 데이터 · 힙 · 스택 /proc/PID/maps addr_show · maps_show	R · S · D · T · Z 상태 전이와 좀비 state_show	FD 표 · /proc/PID/fd status · soft·hard 한계 hold_open · procdump	질문별 관찰 창 할 수 있는 일 7주차 심화로

반복 동작: 프로그램이 스스로 보고한 값 ↔ ps · /proc/PID 가 밖에서 관찰한 값

실행 화면은 iTerm에서 Ubuntu 24.04(x86-64 · 커널 6.8) 실습 서버에 접속해 캡처한 실제 화면입니다.

프로그램과 프로세스



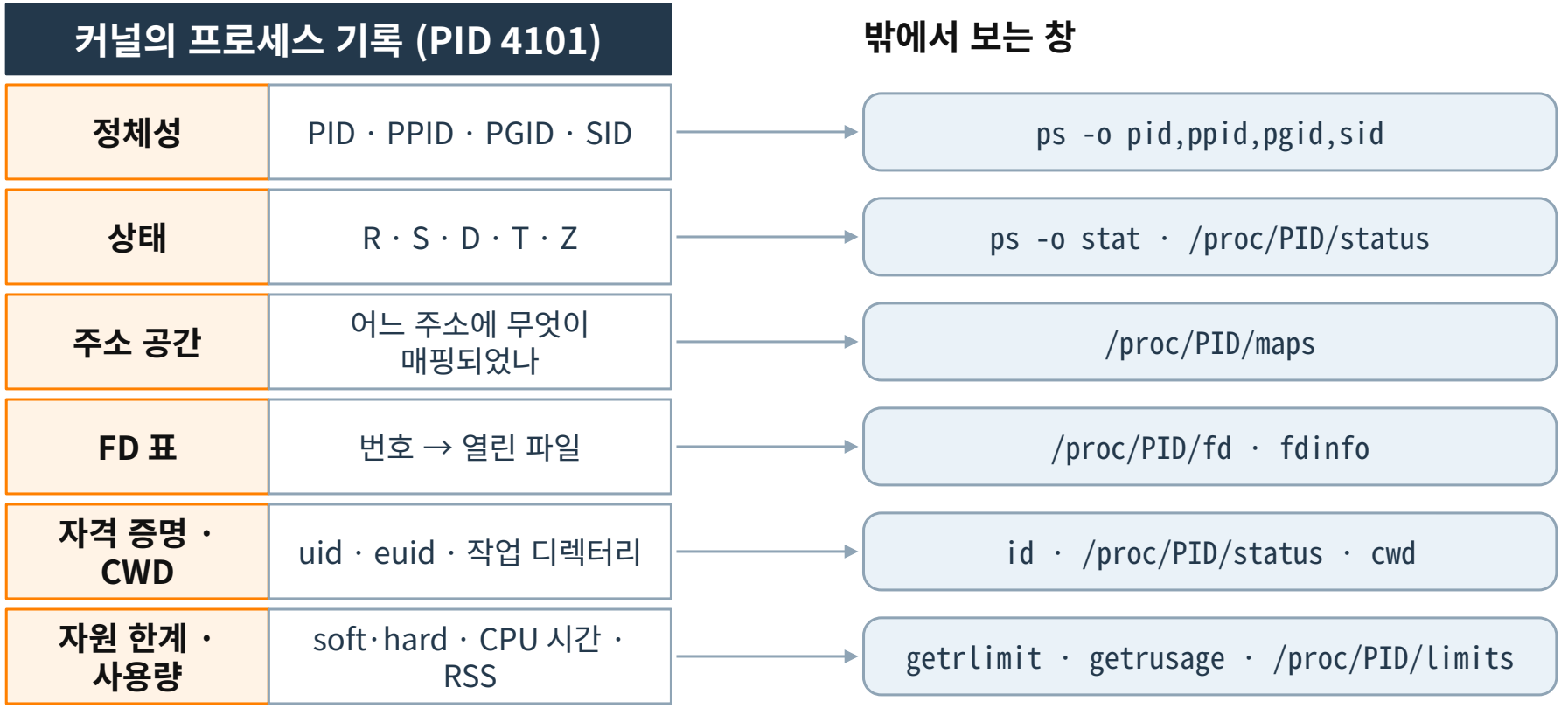
같은 프로그램이어도 PID·상태·자원은 프로세스마다 따로 있다

프로그램은 파일이고, 프로세스는 그 파일을 실행 중인 상태와 자원의 묶음입니다.

커널이 프로세스마다 기록하는 것

6주차 1차시
정체성 · 자격 증명 · CWD를 조회했다.

오늘
상태 · 주소 공간 · FD 표 · 자원 한계·사용량을 관찰한다.

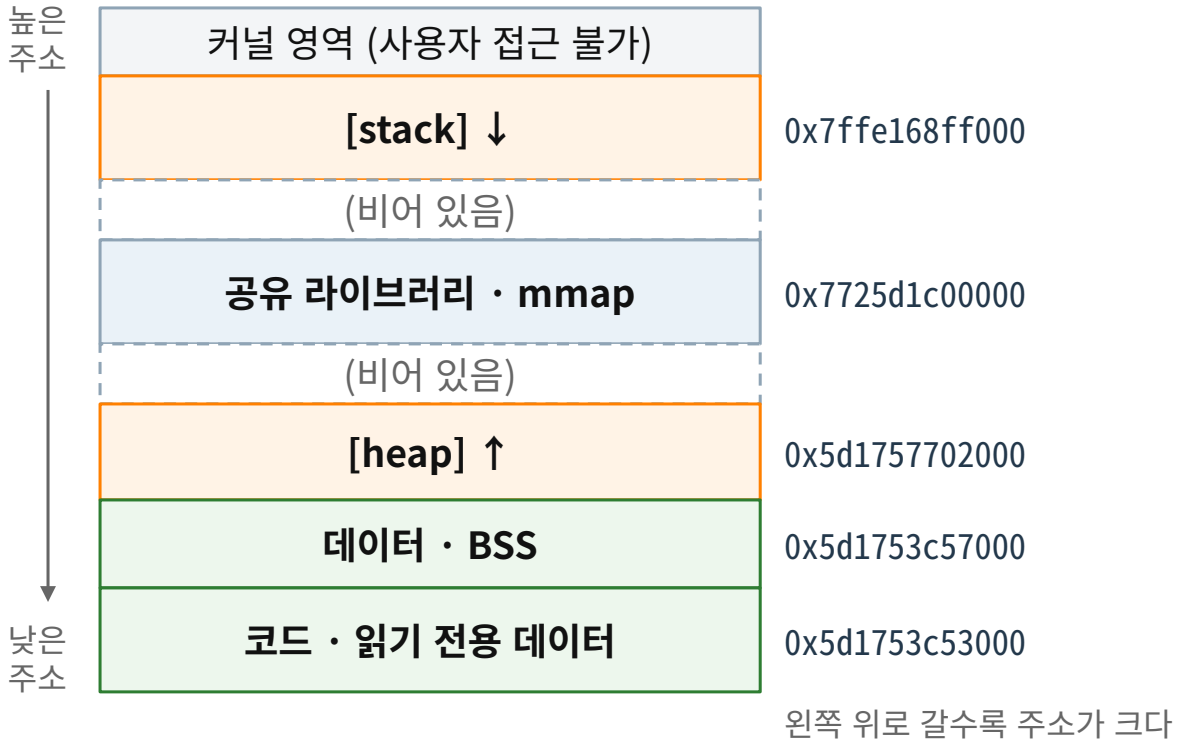


Linux에서는 이 기록을 task_struct라는 커널 구조체가 담고, /proc/PID 디렉터리가 그 창 역할을 합니다.

1부
주소 공간

코드·데이터·힙·스택이 어디에 놓이는지 maps로 확인합니다.

가상 주소 공간의 배치



구역마다 규칙이 다르다

코드는 읽기·실행, 데이터는 쓰기.
힙은 malloc, 스택은 호출이 키운다.

빈 공간이 대부분이다

주소 공간은 넓고 대부분 비었다.
매핑된 곳만 쓸 수 있다.

실행마다 달라진다

ASLR이 시작 위치를 섞는다.
순서는 같고 값은 매번 다르다.

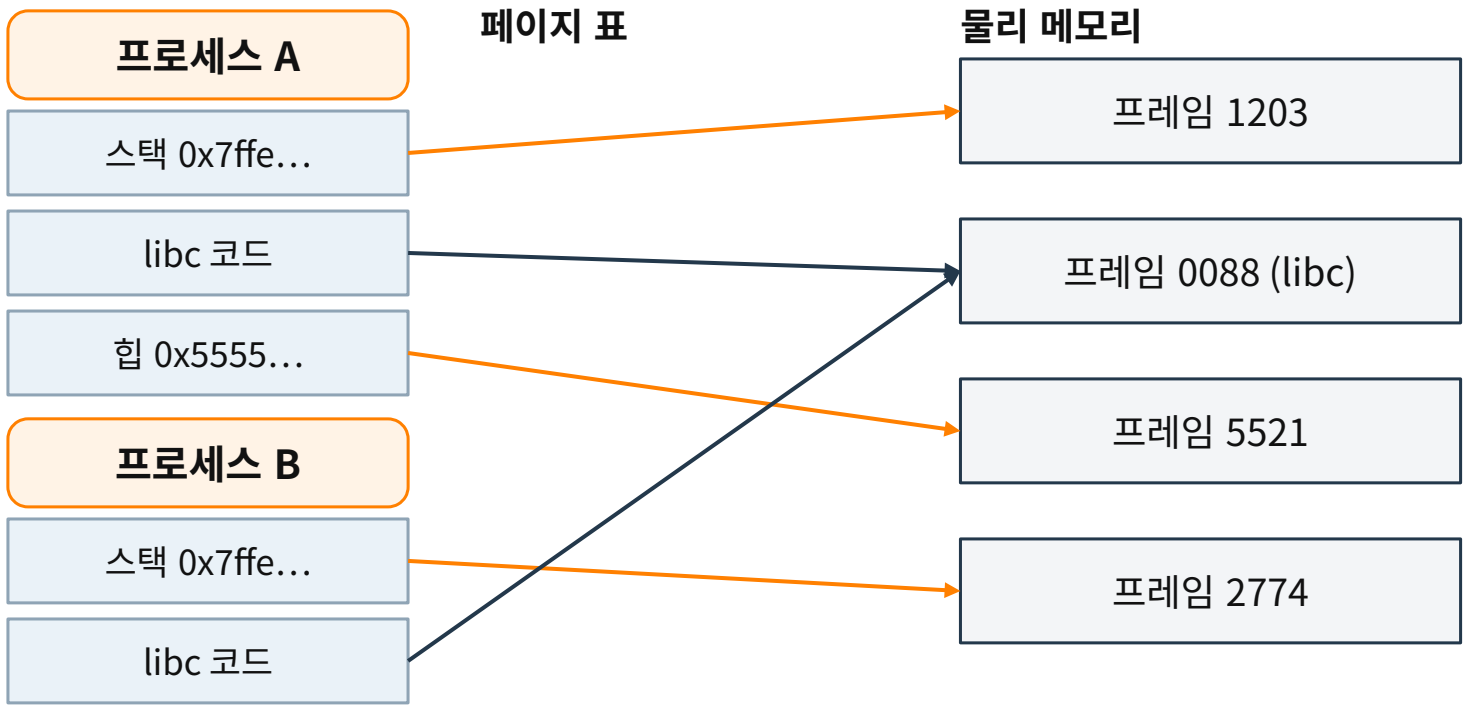
주소는 addr_show 실행(Ubuntu 24.04 · x86-64)에서 관찰한 값입니다. 같은 프로그램도 실행마다 값이 바뀝니다.

변수 선언이 구역을 정한다

`int global_init = 7;`	전역 · 초기값 있음	→ 데이터 (.data)
`int global_zero;`	전역 · 초기값 없음	→ BSS (.bss) · 0으로 시작
`static int local_static = 9;`	함수 안이어도 static	→ 데이터 (.data)
`int local = 3;`	함수 안의 지역 변수	→ 스택 · 함수 프레임
`char *heap1 = malloc(64);`	heap1이 가리키는 블록	→ 힙 · free까지 유지
`int main(void) { ... }`	함수의 기계어	→ 코드 (.text) · 읽기·실행

저장 기간(storage duration)이 구역을 정합니다. 정적 저장 기간은 데이터·BSS, 자동은 스택, 동적은 힙입니다.

가상 주소와 물리 메모리



A와 B의 스택은 같은 가상 주소대여도 다른 프레임 · libc 코드는 한 프레임을 함께 쓴다

가상 주소

프로그램이 보는 주소다.
프로세스마다 자기 공간을 가진다.

페이지 표

4 KiB 페이지 단위로 가상 →
물리 프레임 대응을 기록한다.

공유와 격리

같은 가상 주소도 다른 프레임이다.
읽기 전용 코드는 공유할 수 있다.

%p로 출력한 주소는 모두 가상 주소입니다. 물리 주소는 사용자 프로그램이 볼 수 없습니다.

addr_show: 구역별 주소 출력

```
int global_init = 7;            /* 데이터 */
int global_zero;                /* BSS */

int local = 3;                  /* 스택 */
static int local_static = 9;    /* 데이터 */
char *heap1 = malloc(64);       /* 힙 */

printf("data=%p bss=%p\n",
       (void *)&global_init, (void *)&global_zero);
printf("data_static=%p stack=%p\n",
       (void *)&local_static, (void *)&local);
printf("heap1=%p heap2=%p\n",
       (void *)heap1, (void *)heap2);
```

완성 예제: labs/class07-01/addr_show.c 9–44행

%p와 (void *)

포인터 출력 서식 %p는
void *를 받는다. 변환해서 넘긴다.

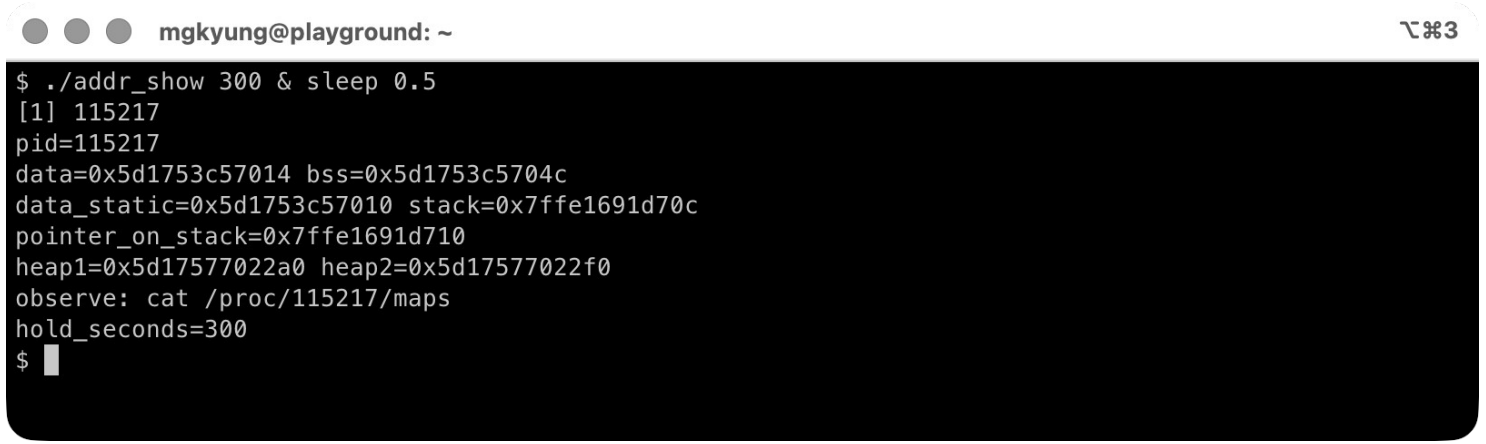
관찰 시간

[seconds] 인자만큼 살아 있어
같은 PID의 maps를 읽을 수 있다.

비교하지 않는 것

서로 다른 객체의 주소 대소
비교는 C에서 정의되지 않는다.

addr_show 실행 화면



세 무리

0x5d1753c57…(데이터·BSS),
0x5d175770…(힙), 0x7ffe…(스택).

힙 두 블록

heap2 − heap1 = 0x50 = 80.
64바이트 + 할당자 관리 정보.

포인터 변수

heap1 변수 자체(&heap1)는
스택에 있다. 0x7ffe…로 시작한다.

iTerm · Ubuntu 24.04 실습 서버 실제 화면 · 이 프로세스(PID 115217)의 maps를 다음 장들에서 읽습니다.

maps 한 행 읽기

5d1753c57000-5d1753c58000	rw-p	00003000	fc:00	2229087	/home/.../addr_show
주소 범위	**권한**	**오프셋**	**장치**	**아이노드**	**경로**
시작 이상 · 끝 미만 여기서는 0x1000 = 4 KiB	r 읽기 · w 쓰기 · x 실행 p 사유 · s 공유	파일의 어느 위치를 매핑했나	파일이 있는 장치 번호	파일 번호 (익명이면 0)	매핑한 파일 또는 [heap]·[stack]

읽는 법: addr_show 파일의 오프셋 0x3000부터를 0x5d1753c57000에 4 KiB 매핑했고, 읽고 쓸 수 있으며 이 프로세스 전용이다.
→ 전역 변수 data(…57014)와 bss(…5704c)가 이 범위 안에 있다.

man 5 proc_pid_maps · 경로가 없는 행은 익명 매핑이고, [heap]·[stack] 같은 이름은 커널이 붙인 표시입니다.

addr_show의 maps 전체

mgkyung@playground: ~ ⌥⌘3

```
$ cat /proc/115217/maps
5d1753c53000-5d1753c54000 r--p 00000000 fc:00 2229087                    /home/mgkyung/w07/class07-01/addr_show
5d1753c54000-5d1753c55000 r-xp 00001000 fc:00 2229087                    /home/mgkyung/w07/class07-01/addr_show
5d1753c55000-5d1753c56000 r--p 00002000 fc:00 2229087                    /home/mgkyung/w07/class07-01/addr_show
5d1753c56000-5d1753c57000 r--p 00002000 fc:00 2229087                    /home/mgkyung/w07/class07-01/addr_show
5d1753c57000-5d1753c58000 rw-p 00003000 fc:00 2229087                    /home/mgkyung/w07/class07-01/addr_show
5d1757702000-5d1757723000 rw-p 00000000 00:00 0                          [heap]
7725d1c00000-7725d1c28000 r--p 00000000 fc:00 2387791                    /usr/lib/x86_64-linux-gnu/libc.so.6
7725d1c28000-7725d1db1000 r-xp 00028000 fc:00 2387791                    /usr/lib/x86_64-linux-gnu/libc.so.6
7725d1db1000-7725d1e00000 r--p 001b1000 fc:00 2387791                    /usr/lib/x86_64-linux-gnu/libc.so.6
7725d1e00000-7725d1e04000 r--p 001ff000 fc:00 2387791                    /usr/lib/x86_64-linux-gnu/libc.so.6
7725d1e04000-7725d1e06000 rw-p 00203000 fc:00 2387791                    /usr/lib/x86_64-linux-gnu/libc.so.6
7725d1e06000-7725d1e13000 rw-p 00000000 00:00 0
7725d1fc8000-7725d1fcb000 rw-p 00000000 00:00 0
7725d1fd5000-7725d1fd7000 rw-p 00000000 00:00 0
7725d1fd7000-7725d1fd8000 r--p 00000000 fc:00 2387625                    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7725d1fd8000-7725d2003000 r-xp 00001000 fc:00 2387625                    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7725d2003000-7725d200d000 r--p 0002c000 fc:00 2387625                    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7725d200d000-7725d200f000 r--p 00036000 fc:00 2387625                    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7725d200f000-7725d2011000 rw-p 00038000 fc:00 2387625                    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffe168ff000-7ffe16920000 rw-p 00000000 00:00 0                          [stack]
7ffe16990000-7ffe16994000 r--p 00000000 00:00 0                          [vvar]
7ffe16994000-7ffe16996000 r-xp 00000000 00:00 0                          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                  [vsyscall]
$
```

위 1–5행 · 실행 파일
헤더 · 코드(r-xp) · 상수 · RELRO · 데이터(rw-p)

6행 · [heap]
brk로 늘리는 힙 · 0x21000 = 132 KiB

7–11행 · libc
실행 파일과 같은 다섯 갈래 구성

12–14행 · 이름 없는 rw-p
라이브러리 BSS · 스레드 지역 저장소 등

15–19행 · 동적 링커 ld.so
실행 직후 라이브러리를 적재하는 프로그램

20행 · [stack]
주 스레드의 스택 · 0x21000 = 132 KiB

21–23행 · vvar 등
커널이 넣어 준 빠른 호출 영역 (vdso) · 옛 호환 영역(vsyscall)

iTerm · cat /proc/115217/maps 실제 화면 (앞 장의 addr_show 프로세스)

출력한 주소를 maps 구역에 넣기

addr_show 출력		그 주소를 포함하는 maps 행
data	0x5d1753c57014	5d1753c57000-5d1753c58000 rw-p addr_show
bss	0x5d1753c5704c	5d1753c57000-5d1753c58000 rw-p addr_show
data_static	0x5d1753c57010	5d1753c57000-5d1753c58000 rw-p addr_show
heap1	0x5d17577022a0	5d1757702000-5d1757723000 rw-p [heap]
stack	0x7ffe1691d70c	7ffe168ff000-7ffe16920000 rw-p [stack]

판정 규칙: 시작 ≤ 주소 < 끝. 16진수 앞자리부터 맞춰 보면 빠릅니다.

권한 문자로 구역 알아보기

r-xp: 코드

읽고 실행할 수 있고 쓸 수 없다.
코드에 쓰면 SIGSEGV로 끝난다.

r--p: 읽기 전용 데이터

문자열 상수와 재배치가 끝난
주소 표(RELRO)가 여기에 놓인다.

rw-p: 데이터·힙·스택

쓸 수 있는 구역이다. p는 쓰기가
이 프로세스 사본에만 반영된다는 뜻.

---p 처럼 권한이 하나도 없는 행은 경계용 보호 구역입니다.

힙과 스택의 수명

스택 (호출 순서대로 쌓임)

main 프레임

parse 프레임

helper 프레임 ← 반환하면 사라짐

지역 변수 int local → 프레임 안
helper가 반환한 뒤 그 주소는 무효

힙 (free까지 남음)

블록 A 64 B · parse가 malloc

블록 B 64 B · helper가 malloc

(free된 자리 · 재사용 가능)

helper가 끝나도 블록 B는 남는다
포인터를 잃으면 누수가 된다

스택 프레임: 호출 → 생성, 반환 → 소멸
힙 블록: malloc → 생성, free → 소멸 (함수 경계와 무관)

스택: 자동 수명

함수가 반환하면 프레임이 사라진다.
지역 변수의 주소를 돌려주면 안 된다.

힙: 명시적 수명

free할 때까지 남는다. 잊으면 누수,
두 번 free하면 정의되지 않은 동작.

크기 한계

스택은 RLIMIT_STACK(보통 8 MiB).
깊은 재귀는 스택 한계에서 죽는다.

스택은 함수 호출이, 힙은 malloc·free가 크기를 바꿉니다.

예약한 크기와 올라와 있는 크기

VmSize 2684 kB (매핑한 범위 전체)

VmRSS 1800 kB (올라와 있음)	아직 안 건드림

VmExe	VmData	VmStk	VmLib
4 kB 실행 파일 코드	224 kB 데이터·힙	132 kB 스택	1752 kB 공유 라이브러리

작은 C 프로그램도 VmLib(libc)이 대부분이다.
libc 페이지는 다른 프로세스와 공유되므로 RSS를 그냥 더하면
시스템 전체 사용량보다 커진다.

VmSize (VSZ)

매핑한 가상 주소 범위의 합이다.
예약일 뿐 메모리 사용이 아니다.

VmRSS (RSS)

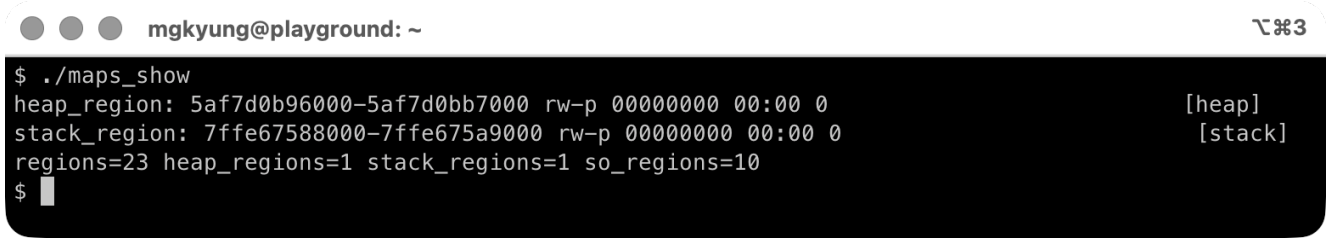
지금 물리 메모리에 올라와 있는
페이지의 합이다. 공유 페이지 포함.

요구 페이징

페이지는 처음 건드릴 때 올라온다.
그래서 VmRSS ≤ VmSize다.

state_show 실행 중 /proc/PID/status 실제 값(36쪽 화면) · 측정 실험은 7주차 2차시 pagetouch에서 합니다.

maps_show: /proc/self/maps 세기




```c
FILE *fp = fopen("/proc/self/maps", "r");
while ((len = getline(&line, &cap, fp)) != -1) {
    total++;
    if (strstr(line, "[heap]") != NULL) ...      /* 행 출력, heap++ */
    else if (strstr(line, "[stack]") != NULL) ... /* 행 출력, stack++ */
    else if (strstr(line, ".so") != NULL) so++;
}
```

self

/proc/self는 읽는 프로세스 자신을 가리키는 링크다.

getline

길이 제한 없이 한 행씩 읽고 버퍼는 마지막에 한 번 free한다.

so_regions=10

libc 5행 + ld.so 5행이다. 라이브러리 개수(2개)가 아니다.

iTerm · Ubuntu 24.04 실습 서버 실제 화면 · 완성 예제: labs/class07-01/maps_show.c

직접 확인 1: 주소 공간

```
./addr_show 30 &
cat /proc/$!/maps              # 출력한 주소가 어느 행에?
grep -E 'heap|stack' /proc/$!/maps
grep -E '^Vm' /proc/$!/status
wait

./maps_show
./maps_show; ./maps_show       # 주소가 바뀌는가?
ulimit -s                      # 스택 한계 (KiB)
```

기록은 labs/class07-01/REPORT.md의 실습 1 칸에 합니다.

대응표

data·bss·heap1·stack 주소를 포함하는 maps 행을 찾아 적는다.

두 번 실행

ASLR 때문에 값은 달라지고 구역 순서는 같다는 것을 본다.

숫자 셋

VmSize · VmRSS · VmStk를 적고 서로의 관계를 설명한다.

2부
상태

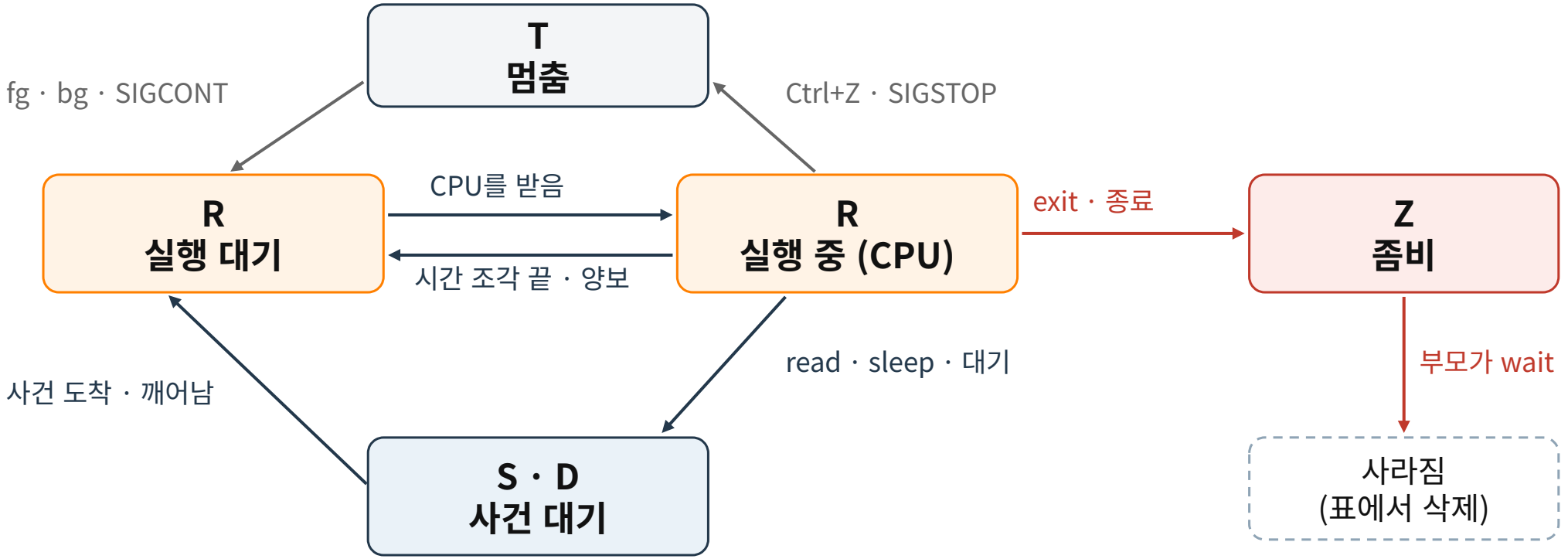
R·S·D·T·Z 상태 문자와 전이를 관찰합니다.

상태 문자 한눈에

문자	뜻	대표 원인	벗어나는 방법
R	실행 중 · 실행 대기	계산 루프, CPU 차례를 기다림	스케줄러가 CPU를 준다
S	잠자기 (깨울 수 있음)	sleep, 키보드·소켓·파이프 대기	기다리던 사건이나 시그널
D	깨울 수 없는 잠	디스크·네트워크 파일시스템 I/O	I/O 완료 (보통 아주 짧다)
T	멈춤 (작업 제어)	Ctrl+Z, SIGSTOP	fg · bg · SIGCONT
t	멈춤 (추적 중)	디버거의 중단점	디버거가 계속 실행
Z	좀비 (종료 · 회수 전)	부모가 아직 wait하지 않음	부모의 wait (회수)
I	유휴 커널 스레드	할 일 없는 커널 작업자	커널이 일을 맡김

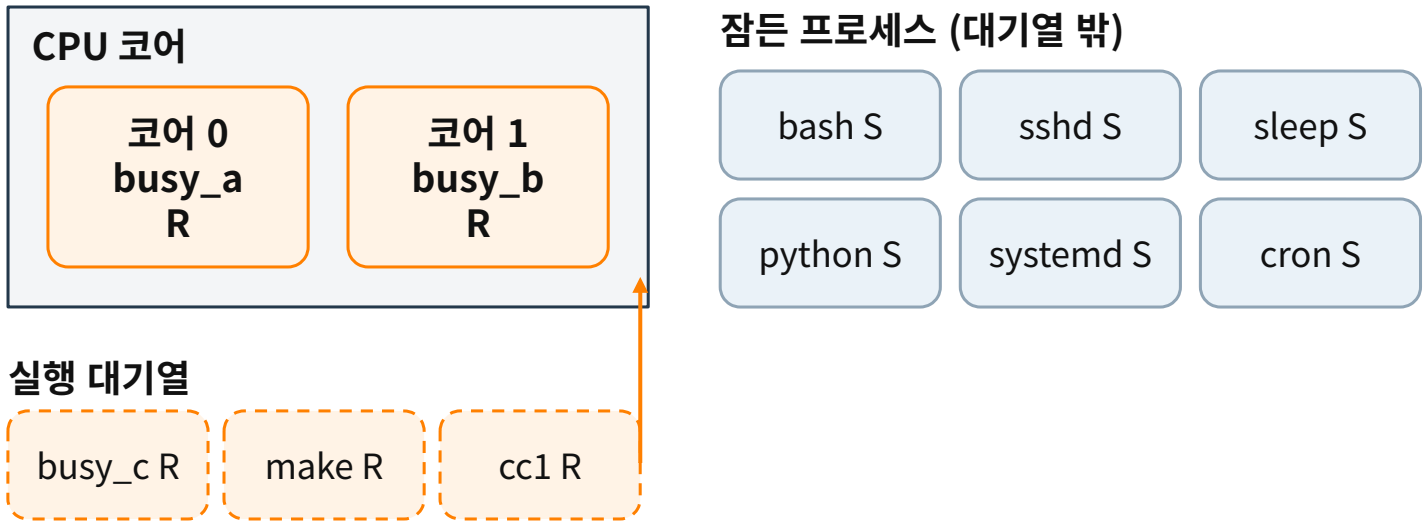
ps의 STAT 첫 글자와 /proc/PID/status의 State 줄이 같은 상태를 보여 줍니다.

상태 전이 지도



Ctrl+C는 종료(→ Z), Ctrl+Z는 멈춤(→ T)입니다. 시그널의 정식 규칙은 9주차에 다룹니다.

R: 실행 중과 실행 대기



ps에서 R이 5개 = 2개는 CPU에서 실행 중 + 3개는 차례를 기다림

R은 둘을 묶는다

CPU에서 도는 중과 차례를 기다리는 중을 구분하지 않는다.

코어 수보다 많을 수 있다

R이 코어 수보다 많으면 누군가는 줄을 서 있다.

S는 줄 밖에 있다

잠든 프로세스는 CPU를 원하지 않는다. 사건이 오면 줄에 선다.

실습 서버는 48코어입니다. 수백 개 프로세스 중 R은 보통 몇 개뿐입니다.

S와 D: 두 가지 잠

S: 깨울 수 있는 잠

sleep, 키보드 입력, 소켓·파이프 대기.
시그널이 오면 깨어나 처리할 수 있다.

D: 깨울 수 없는 잠

디스크·네트워크 파일시스템 I/O 중
짧게 들어간다. kill로도 깨지 않는다.

어디서 자는가

ps -o wchan은 잠든 커널 함수를
보여 준다. 예: hrtimer_nanosleep.

ps -o pid,stat,wchan:20,command -p PID 로 잠든 위치를 봅니다.

STAT 열 해독

Ss	S 잠자기
	s 세션 대표 (로그인 셸)
Sl	S 잠자기
	l 멀티스레드
SN	S 잠자기
	N 낮은 우선순위 (nice 양수)
R+	R 실행
	+ 포그라운드 프로세스 그룹
S<	S 잠자기
	< 높은 우선순위 (nice 음수)
T	T 멈춤
	(덧붙임 없음)

첫 글자가 상태이고 뒤의 글자는 덧붙임 정보입니다. 덧붙임 글자는 BSD 형식 출력(ps -o stat, ps aux)에만 붙습니다.

state_show: 상태 관찰 대상 만들기

```
for (long done = 1; done <= seconds; done++) {
    double next = start + (double)done;
    if (busy) {
        while (now_sec() < next) {
            /* R 관찰용: CPU를 놓지 않는다 */
        }
    } else {
        sleep(1);           /* S 관찰용: 1초씩 잠든다 */
    }
    printf("tick=%ld of %ld\n", done, seconds);
}
```

완성 예제: labs/class07-01/state_show.c 55–70행

두 모드

기본은 잠자기(S),
--busy는 계산 루프(R)다.

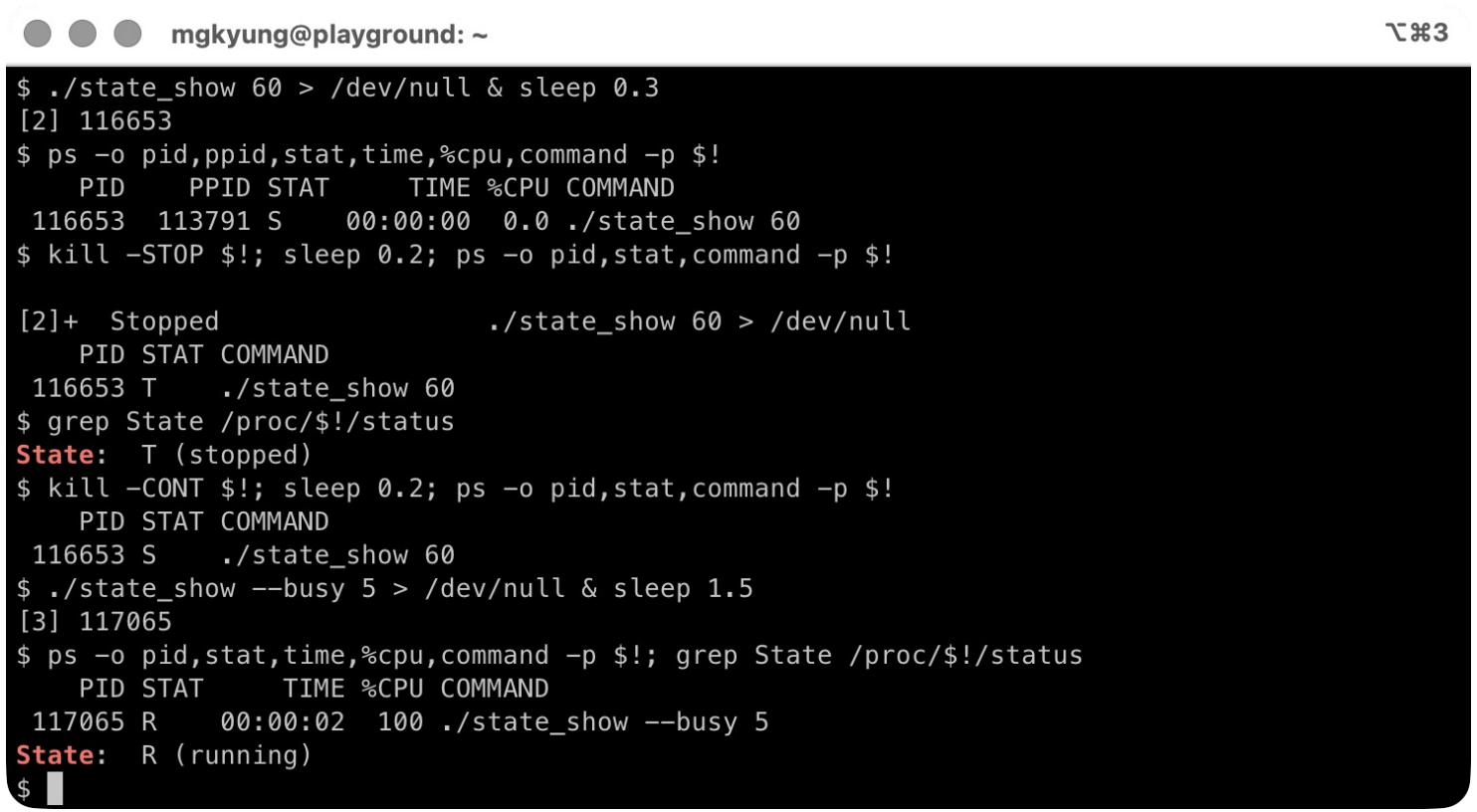
마감 시각

busy 루프는 CLOCK_MONOTONIC
기준 1초 마감까지 돈다.

관찰 안내

시작할 때 비교할 ps 명령과
/proc 경로를 스스로 출력한다.

S·R·T를 만든 실제 화면



iTerm · Ubuntu 24.04 실습 서버 실제 화면

S → T → S

SIGSTOP으로 멈추고 SIGCONT로 재개했다. Ctrl+Z·fg와 같은 전이.

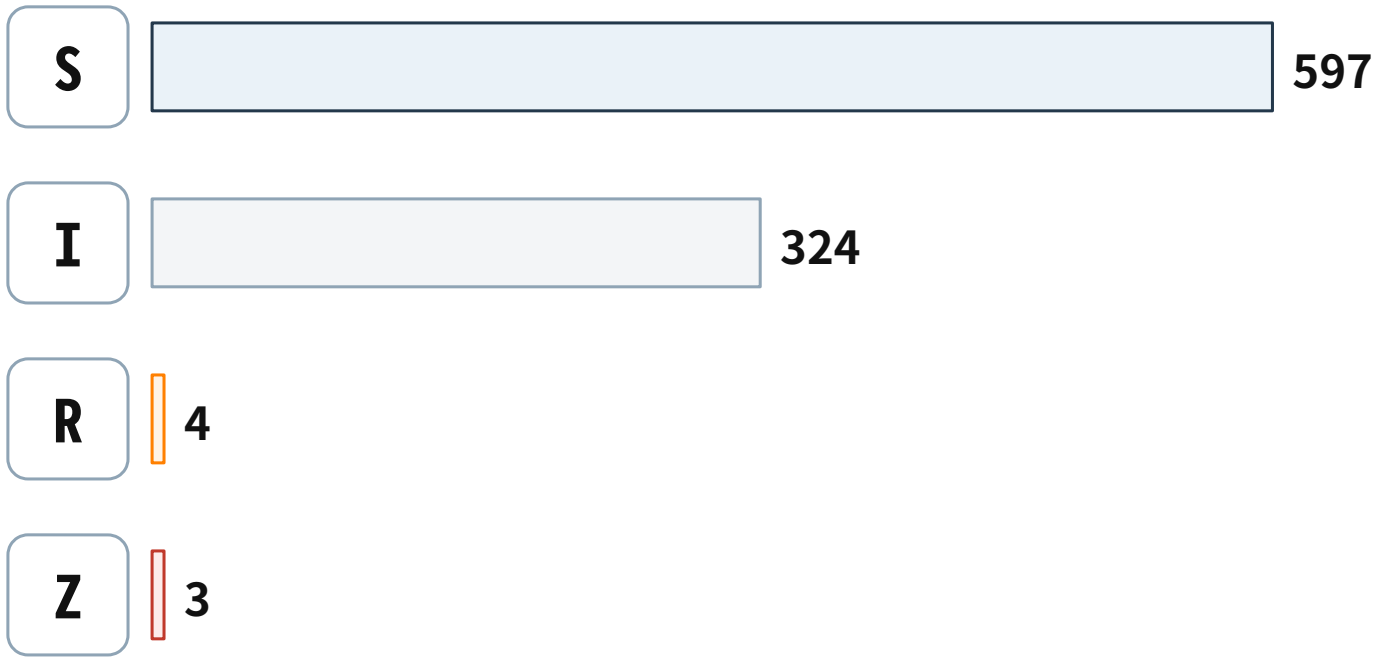
R과 %CPU

busy는 R이고 %CPU가 100이다. 셸은 멈춘 작업을 Stopped로 알린다.

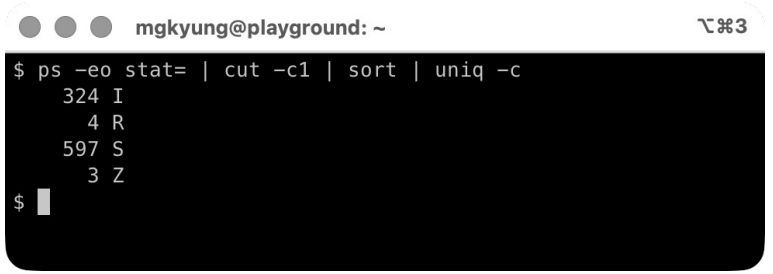
두 창 일치

ps의 STAT와 /proc의 State가 같은 글자를 보여 준다.

한 서버의 상태 분포



전체 928개 프로세스 · I는 커널 유휴 스레드



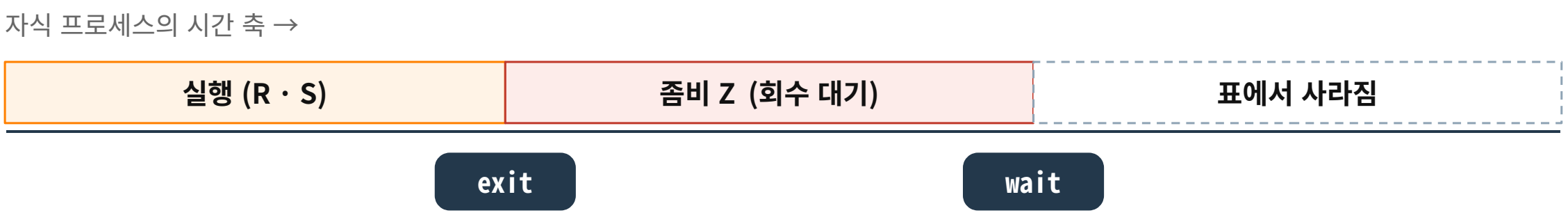
대부분은 잔다: 서버 프로세스는 요청이 올 때까지 기다린다.

R은 몇 개뿐: 그중 하나는 ps 자신이다. 48코어 중 대부분이 놀고 있다.

Z 3개: 회수되지 않은 좀비다. PPID로 부모를 찾아볼 수 있다.

iTerm · ps -eo stat= | cut -c1 | sort | uniq -c 실제 화면 (Ubuntu 24.04 실습 서버)

좀비: 종료와 회수 사이



exit 즉시 반납

주소 공간(메모리) · 열린 FD 전부
작업 디렉터리 참조 등

좀비가 남기는 것

PID · 종료 상태(예: 7)
CPU 시간 등 사용량 통계

좀비는 모든 프로세스가 거치는 단계이고, 오래 남는 것이 문제입니다. 실습은 7주차 2차시 reap_show에서.

잠자기와 좀비는 다르다

S: 살아 있다

메모리·FD를 그대로 들고 사건을
기다린다. 깨어나면 일을 계속한다.

Z: 이미 끝났다

자원은 반납했고 종료 상태만
남았다. 어떤 시그널에도 반응 없다.

구별법

STAT 첫 글자, ps의 <defunct> 표시,
VmRSS 줄 유무(좀비는 없다).

S는 사건이 깨우고 · Z는 부모가 치웁니다

직접 확인 2: 상태

```
./state_show 60 &                    # S
ps -o pid,ppid,stat,time,%cpu,command -p $!
./state_show --busy 20 &             # R
ps -o pid,stat,%cpu,command -p $!
kill %2

./state_show 60                      # 실행 중 Ctrl+Z → T
ps -o pid,stat,command -p <PID>
fg                                   # 재개 → S, 끝까지 기다림

ps -o pid,stat,wchan:20,command -p <PID>
ps -eo stat= | cut -c1 | sort | uniq -c
```

기록은 REPORT.md의 실습 2 칸에 합니다.

세 상태

S · R · T를 직접 만들고
STAT와 State를 함께 적는다.

잠든 위치

wchan 열로 어느 커널 함수에서
잠들었는지 본다.

정리

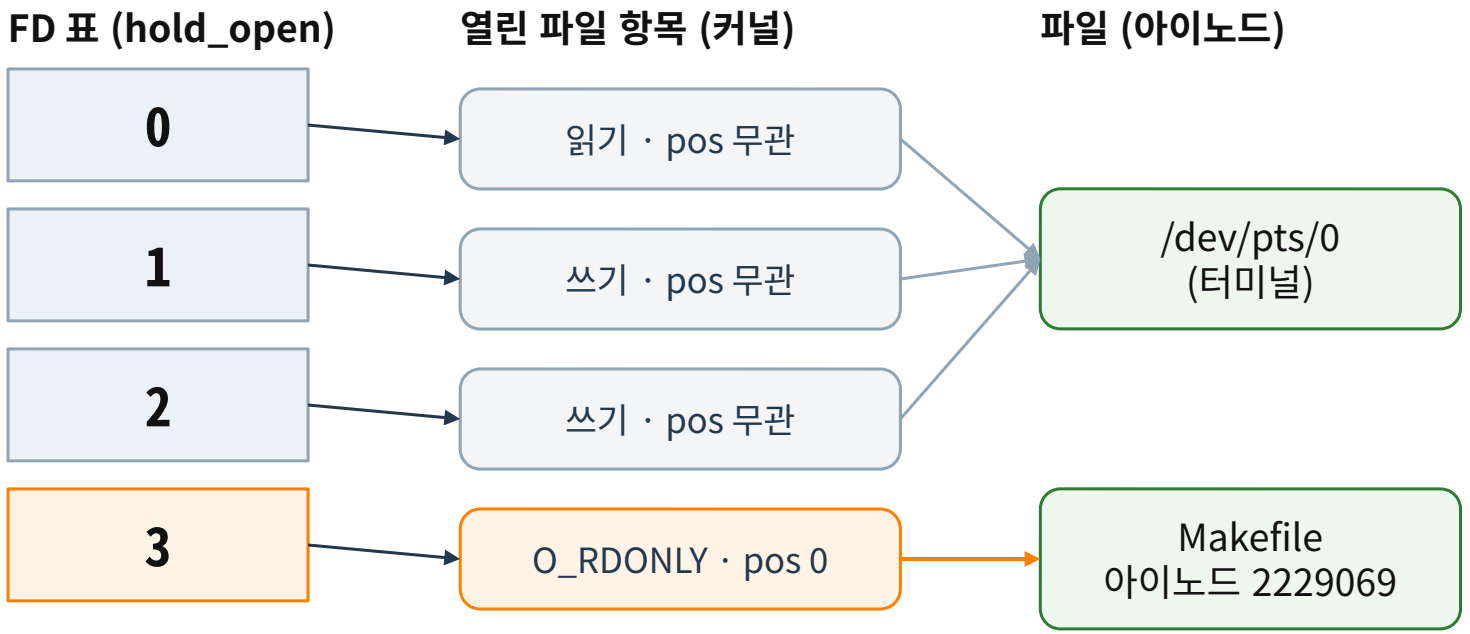
백그라운드 작업은 jobs로
확인하고 남기지 않는다.

3부

자원

FD 표, /proc/PID/status, 자원 한계를 관찰합니다.

FD 표의 세 층



0·1·2는 보통 같은 터미널을 가리킨다 · 3은 hold_open이 연 Makefile

FD 표 (프로세스별)

작은 정수 → 열린 파일 항목.
open은 가장 작은 빈 번호를 준다.

열린 파일 항목

파일 위치(pos)와 열기 플래그를 담는다. open 한 번에 하나.

아이노드

실제 파일이다. 여러 항목이 같은 파일을 가리킬 수 있다.

3주차의 FD를 오늘은 밖에서 관찰합니다. dup·fork로 항목을 공유하는 규칙은 10·12주차에 다시 만납니다.

hold_open: FD 관찰 대상 만들기

```
int fd = open(argv[1], O_RDONLY);
if (fd == -1) {
    perror(argv[1]);
    return 1;
}
printf("pid=%ld fd=%d path=%s\n",
       (long)getpid(), fd, argv[1]);
for (long done = 1; done <= seconds; done++) {
    sleep(1);                    /* 관찰할 시간 창 */
    printf("hold=%ld of %ld\n", done, seconds);
}
if (close(fd) == -1) {
    perror("close");
    return 1;
}
```

완성 예제: labs/class07-01/hold_open.c 27–53행

번호 예측

0·1·2가 열려 있으면
첫 open은 3을 돌려준다.

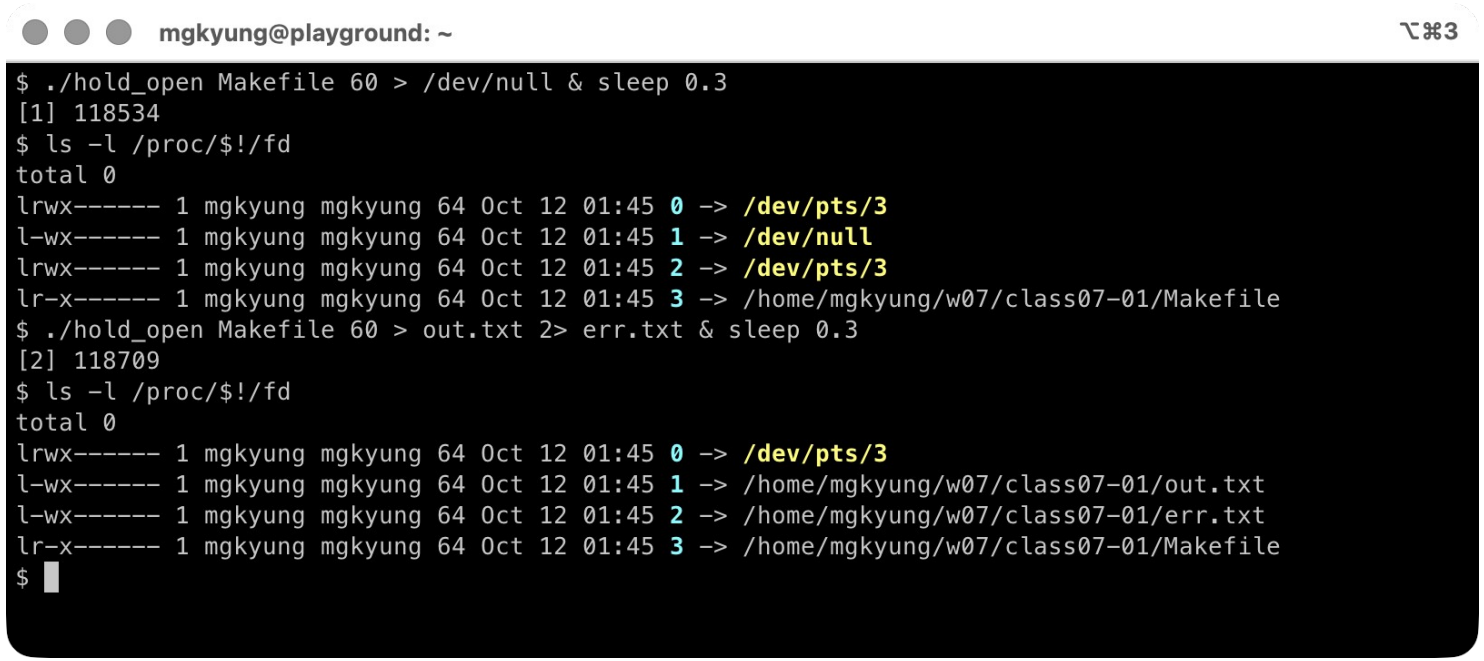
시간 창

열고 닫는 사이에 다른 셸에서
/proc/PID/fd를 본다.

close 검사

close도 실패할 수 있다.
결과를 확인하는 것이 원칙이다.

/proc/PID/fd로 본 FD 표



심볼릭 링크

각 번호가 무엇을 가리키는지
링크 대상으로 보여 준다.

리다이렉션의 정체

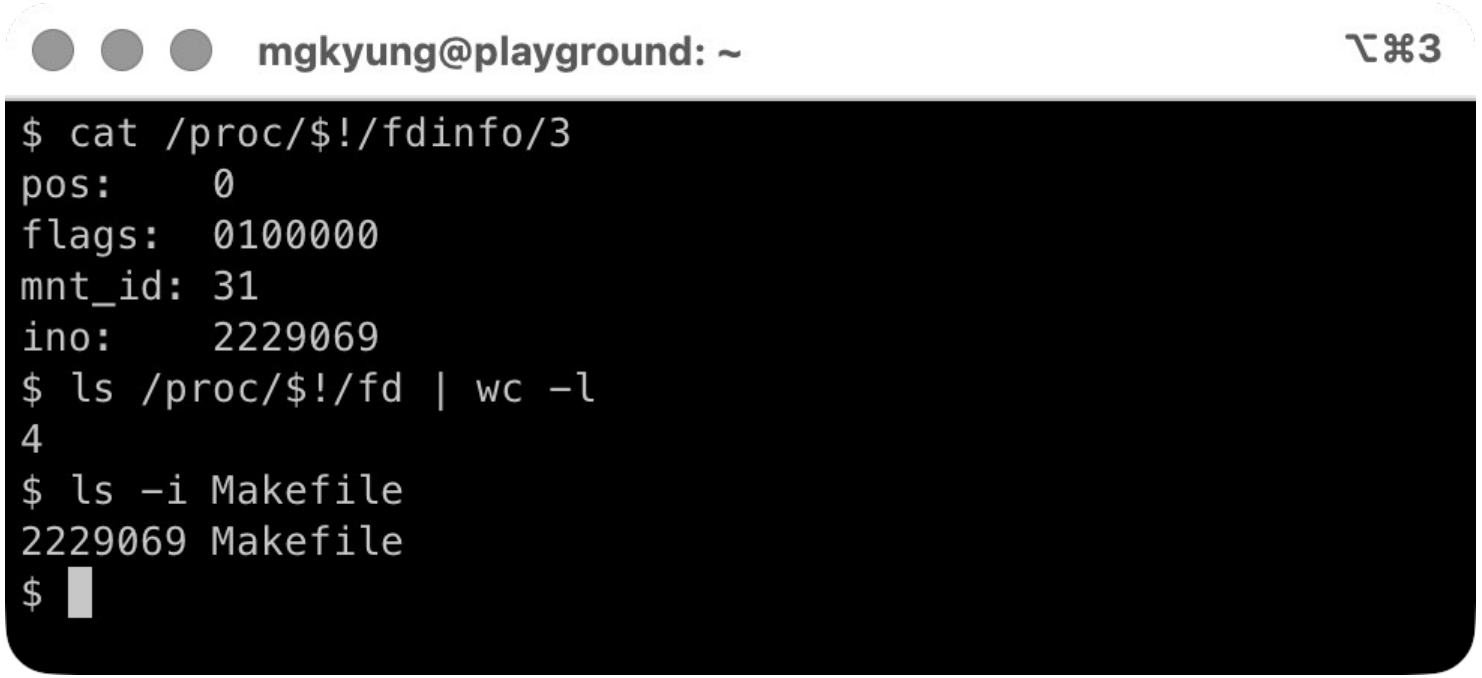
셸이 실행 전에 FD 1·2를
파일로 바꿔 둔 것이다.

프로그램은 모른다

hold_open 코드는 그대로다.
가리키는 대상만 달라졌다.

iTerm · Ubuntu 24.04 실습 서버 실제 화면 · lr-x 읽기 전용 · l-wx 쓰기 전용 · lrwx 읽기·쓰기로 연 FD

fdinfo: 열린 파일 항목 엿보기



pos

다음 read가 시작할 파일 위치다.
아직 읽지 않아 0이다.

flags (8진수)

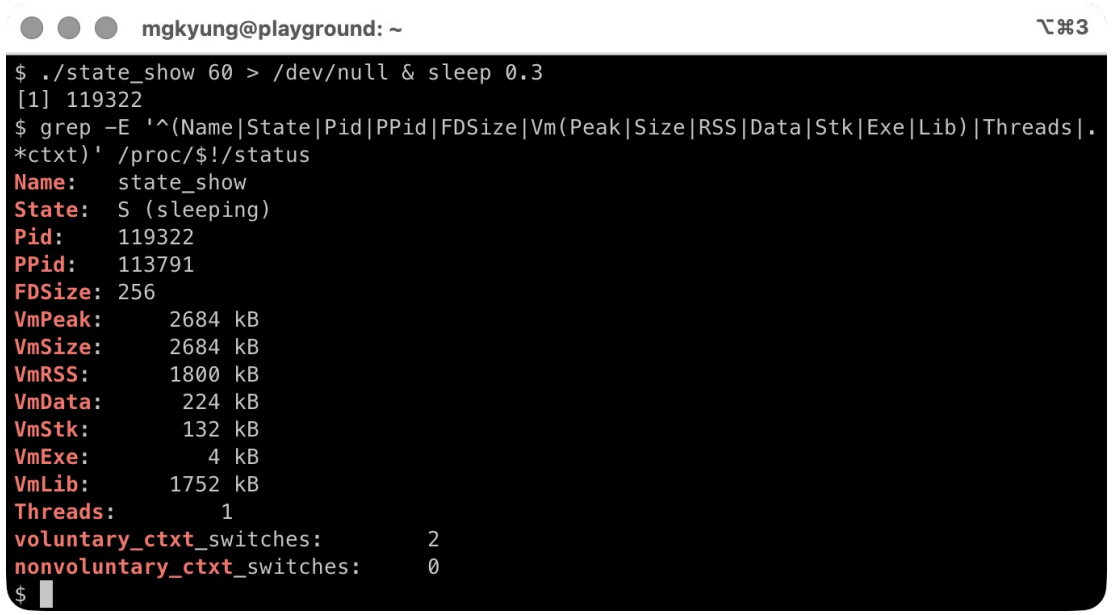
0100000은 O_LARGEFILE이다.
접근 모드 비트 0 = O_RDONLY.

ino

Makefile의 아이노드 번호다.
ls -i Makefile과 같다(2229069).

iTerm · Ubuntu 24.04 실제 화면 · 플래그 값은 아키텍처마다 다를 수 있습니다.

/proc/PID/status 읽기



① 위 4줄 · 정체성 · 상태
실행 파일 이름(최대 15자), 상태 문자, PID와 부모 PID

② FDSize
FD 표에 마련된 칸 수 (열린 개수가 아니다)

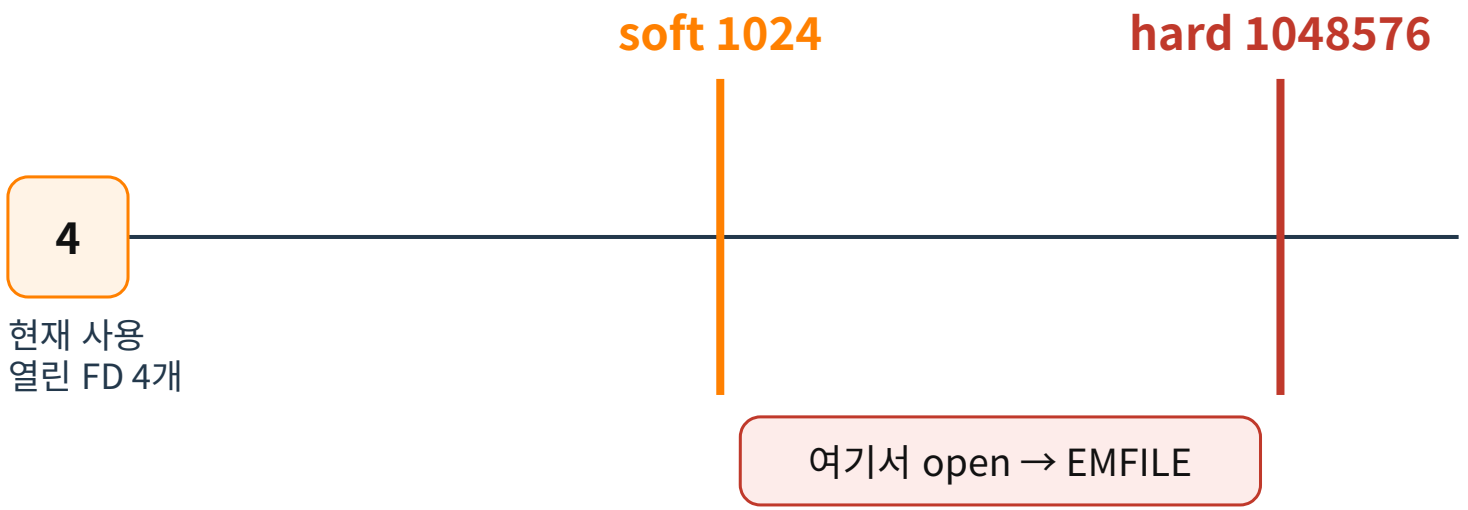
③ Vm으로 시작하는 7줄 · 메모리
최고·현재 예약(VmPeak · VmSize), 상주(VmRSS),
구역별 크기(데이터 · 스택 · 코드 · 라이브러리)

④ Threads
스레드 수 · 1이면 단일 스레드

⑤ 마지막 2줄 · 문맥 교환
스스로 양보한 횟수 · 빼앗긴 횟수 (7주차 2차시에 측정)

iTerm · Ubuntu 24.04 실습 서버 실제 화면 · 형식은 커널 버전에 따라 늘어날 수 있습니다.

자원 한계: soft와 hard



soft

지금 적용되는 한계다. 넘으면 호출이 실패하거나 시그널이 온다.

hard

soft의 천장이다. 일반 사용자는 낮출 수만 있고 되돌릴 수 없다.

상속

한계는 자식에게 그대로 물려진다. 셸의 ulimit가 그 예다.

RLIMIT_NOFILE 실제 값: soft 1024 · hard 1048576 (Ubuntu 24.04 실습 서버, procdump 출력)

procdump: 한 보고서로 묶기

```
show_rlimit(RLIMIT_NOFILE, "rlimit_nofile");
show_rlimit(RLIMIT_AS, "rlimit_as");

struct rusage ru;
if (getrusage(RUSAGE_SELF, &ru) == -1) {
    perror("getrusage");
    return 1;
}
/* 표준 API 다음에 Linux 창으로 보충 */
FILE *fp = fopen("/proc/self/status", "r");
DIR *dir = opendir("/proc/self/fd");
```

완성 예제: labs/class07-01/procdump.c 42–93행

표준 API

getrlimit · getrusage는 macOS와 Linux에서 모두 된다.

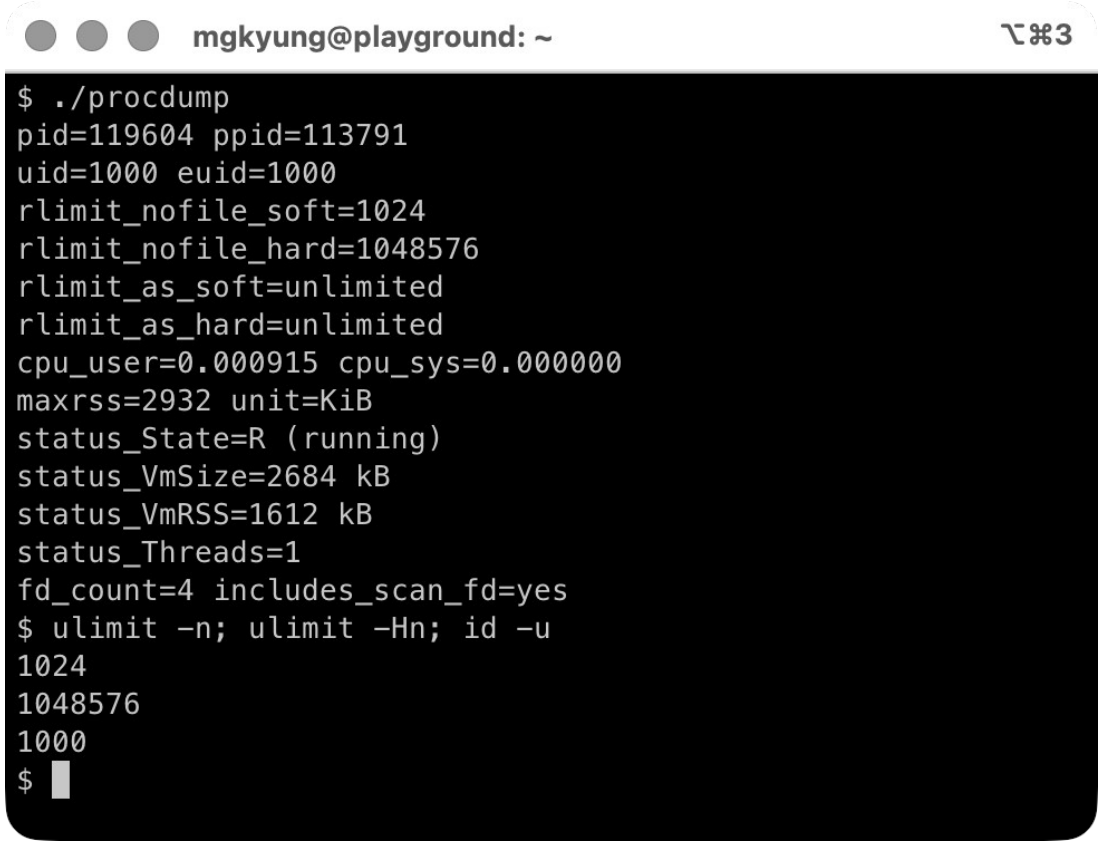
Linux 창

State · VmSize · VmRSS · Threads와 FD 개수를 보충한다.

없으면 알린다

/proc가 없으면 unavailable로 표시하고 계속 진행한다.

procdump 실행 화면



iTerm · Ubuntu 24.04 실습 서버 실제 화면

State가 R

자기 status를 읽는 순간에는 당연히 실행 중이다.

maxrss > VmRSS

maxrss는 최댓값이다. exec 전 셸 사본의 최대 RSS까지 든다.

fd_count=4

0·1·2와 opendir이 연 FD 하나. 관찰 도구도 흔적을 남긴다.

직접 확인 3: 자원

```
./hold_open Makefile 30 &
ls -l /proc/$!/fd
cat /proc/$!/fdinfo/3
ls /proc/$!/fd | wc -l
wait

./hold_open Makefile 30 > out.txt &
ls -l /proc/$!/fd                    # 1은 어디를?
wait

./procdump ; ulimit -n ; ulimit -Hn ; id -u
```

기록은 REPORT.md의 실습 3 칸에 합니다.

FD 대응

출력한 fd 번호와 /proc의 항목, fdinfo의 ino를 맞춘다.

리다이렉션

> out.txt 뒤 FD 1의 대상이 바뀐 것을 기록한다.

한계 일치

procdump의 soft·hard와 ulimit -n · -Hn을 비교한다.

4부

정리

질문별로 어느 창을 볼지 정리합니다.

질문별 관찰 창

질문	프로그램 안 (표준 API)	밖에서 관찰 (ps · /proc)
어디에 무엇이 있나	변수 주소 %p 출력	/proc/PID/maps
지금 무엇을 하나	(직접 조회 API 없음)	ps -o stat,wchan · status의 State
무엇을 열고 있나	open이 돌려준 번호	/proc/PID/fd · fdinfo
메모리를 얼마나 쓰나	getrusage의 ru_maxrss	status의 VmSize · VmRSS
어디까지 쓸 수 있나	getrlimit	ulimit · /proc/PID/limits

판단은 표준 API로, 설명과 진단은 /proc와 ps로 합니다.

정리

주소 공간: 저장 기간이 구역을 정하고, maps 한 행은 범위·권한·파일을 말한다.

메모리 숫자: VmSize는 예약, VmRSS는 지금 올라와 있는 양이다.

상태: R은 실행·대기, S·D는 잠, T는 멈춤, Z는 회수를 기다리는 종료다.

자원: FD는 열린 파일 항목을 가리키는 번호이고, 한계는 soft·hard 한 쌍이다.

확인: 프로그램의 보고를 ps · /proc/PID 와 교차 검증한다.

읽기: man 5 proc_pid_maps · man 5 proc_pid_status · man 1 ps (PROCESS STATE CODES) · 다음: 7주차 2차시 프로세스 심화

이것으로 할 수 있는 일

멈춘 프로그램 진단

STAT이 R이면 계산 루프,
S면 무엇을 기다리는지 wchan을,
D가 길면 저장 장치를 의심한다.

메모리 누수 의심

VmRSS를 시간 간격으로 기록해
계속 늘기만 하는지 본다.
VmSize만 큰 것은 문제가 아니다.

FD 누수 찾기

ls /proc/PID/fd | wc -l 이
계속 늘면 close가 빠진 것이다.
한계에 닿으면 EMFILE.

좀비 원인 찾기

Z의 PPID로 부모를 찾는다.
고칠 것은 자식이 아니라
부모의 회수(wait) 코드다.

리다이렉션 · 로그 확인

서비스의 FD 1·2가 어디로
가는지 /proc/PID/fd로 보면
로그 파일 위치를 알 수 있다.

서비스 한계 점검

/proc/PID/limits로 실행 중인
서버의 NOFILE 한계를 확인하고
동시 연결 수와 비교한다.

관찰이 먼저, 수정은 그다음입니다.

addr_show.c · 1–11행 · 헤더와 파일 주석

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  #include <unistd.h>
04
05  /* Linux의 일반적인 C 실행 환경에서 변수의 가상 주소를 관찰한다.
06     [seconds] 동안 살아 있게 하여 같은 PID의 maps와 대조한다.
07     함수 주소의 void * 변환이나 별개 객체의 포인터 대소 비교는
08     하지 않는다. 코드 영역은 maps의 실행 권한(x)으로 찾는다. */
09  int global_init = 7;
10  int global_zero;
11
```

labs/class07-01/addr_show.c · 총 59행

1–3행 · 헤더

stdio(출력), stdlib(malloc·strtol), unistd(getpid·sleep).

5–8행 · 주석

함수 주소 변환과 별개 객체 비교를 하지 않는 이유.

9–10행 · 전역

초기값 있는 전역은 데이터, 없는 전역은 BSS에 놓인다.

addr_show.c · 12–27행 · 인자 검사

```
12  int main(int argc, char **argv)
13  {
14      long seconds = 0;
15      if (argc > 2) {
16          fprintf(stderr, "usage: %s [seconds 0-600]\n", argv[0]);
17          return 2;
18      }
19      if (argc == 2) {
20          char *end = NULL;
21          seconds = strtol(argv[1], &end, 10);
22          if (end == argv[1] || *end != '\0' ||
23              seconds < 0 || seconds > 600) {
24              fprintf(stderr, "usage: %s [seconds 0-600]\n", argv[0]);
25              return 2;
26          }
27      }
```

labs/class07-01/addr_show.c · 총 59행

14행 · 기본값

인자가 없으면 0초,
즉 출력 후 바로 끝난다.

15–18행 · 개수

인자가 둘 이상이면 사용법
오류로 종료 상태 2.

19–26행 · strtol

숫자 아님, 남은 글자, 범위
밖을 모두 거른다.

addr_show.c · 28–46행 · 변수와 주소 출력

```
28      int local = 3;
29      static int local_static = 9;
30      char *heap1 = malloc(64);
31      char *heap2 = malloc(64);
32      if (heap1 == NULL || heap2 == NULL) {
33          perror("malloc");
34          free(heap1);
35          free(heap2);
36          return 1;
37      }
38      printf("pid=%ld\n", (long)getpid());
39      printf("data=%p bss=%p\n",
40             (void *)&global_init, (void *)&global_zero);
41      printf("data_static=%p stack=%p\n",
42             (void *)&local_static, (void *)&local);
43      printf("pointer_on_stack=%p\n", (void *)&heap1);
44      printf("heap1=%p heap2=%p\n", (void *)heap1, (void *)heap2);
45      printf("observe: cat /proc/%ld/maps\n", (long)getpid());
46      printf("hold_seconds=%ld\n", seconds);
```

labs/class07-01/addr_show.c · 총 59행

28–29행 · 지역과 static

local은 스택, local_static은
함수 안이어도 데이터.

30–37행 · malloc

두 블록을 받고 하나라도
실패하면 둘 다 free 후 종료.

38–46행 · 출력

%p에는 (void *)로 바꿔 넘긴다.
관찰 명령도 함께 출력한다.

addr_show.c · 47–59행 · flush, 대기, 해제

```
47      if (fflush(stdout) == EOF) {
48          perror("stdout");
49          free(heap1);
50          free(heap2);
51          return 1;
52      }
53      unsigned int remaining = (unsigned int)seconds;
54      while (remaining != 0)
55          remaining = sleep(remaining);
56      free(heap1);
57      free(heap2);
58      return 0;
59  }
```

labs/class07-01/addr_show.c · 총 59행

47–52행 · fflush

대기 전에 출력을 내보내야
파이프로 읽는 쪽이 본다.

53–55행 · sleep 반복

sleep은 남은 초를 돌려준다.
0이 될 때까지 다시 잔다.

56–58행 · 정리

두 블록을 free하고
종료 상태 0으로 끝낸다.

maps_show.c · 1–11행 · 헤더와 파일 주석

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  #include <string.h>
04
05  /* /proc/self/maps는 이 프로세스의 가상 주소 공간을 행 단위로 보여
06     준다. 각 행은 "시작-끝 권한 오프셋 장치 아이노드 경로" 형식이다.
07     힙([heap])과 스택([stack]) 구역, 공유 라이브러리(.so) 매핑을
08     골라 표시하고 전체 구역 수를 센다. /proc가 없는 환경에서는
09     unsupported로 알린다. Linux에서 열기에 실패하면 실제 오류를
10     보고한다. .so 매핑 수는 라이브러리 파일의 개수와 다르다. */
11
```

labs/class07-01/maps_show.c · 총 66행

1–3행 · 헤더

string.h는 strstr를,
stdlib.h는 free를 위해.

5–8행 · 형식

maps 한 행의 여섯 칸과
고를 구역을 적었다.

8–10행 · 이식성

/proc가 없으면 unsupported,
Linux에서 실패하면 오류.

maps_show.c · 12–31행 · 열기와 이식성 분기

```
12  int main(int argc, char **argv)
13  {
14      if (argc != 1) {
15          fprintf(stderr, "usage: %s\n", argv[0]);
16          return 2;
17      }
18      FILE *fp = fopen("/proc/self/maps", "r");
19      if (fp == NULL) {
20  #ifdef __linux__
21          perror("/proc/self/maps");
22          return 1;
23  #else
24          printf("maps=unsupported (no /proc on this system)\n");
25          if (fflush(stdout) == EOF) {
26              perror("stdout");
27              return 1;
28          }
29          return 0;
30  #endif
31      }
```

labs/class07-01/maps_show.c · 총 66행

14–17행 · 인자

인자를 받지 않는다.
있으면 종료 상태 2.

18행 · fopen

자기 자신의 maps를
텍스트로 연다.

19–30행 · 분기

Linux면 perror 후 1,
아니면 unsupported 후 0.

maps_show.c · 32–47행 · 한 행씩 읽고 분류

```
32      char *line = NULL;
33      size_t cap = 0;
34      ssize_t len;
35      long total = 0, heap = 0, stack = 0, so = 0;
36      while ((len = getline(&line, &cap, fp)) != -1) {
37          total++;
38          if (strstr(line, "[heap]") != NULL) {
39              heap++;
40              printf("heap_region: %s", line);
41          } else if (strstr(line, "[stack]") != NULL) {
42              stack++;
43              printf("stack_region: %s", line);
44          } else if (strstr(line, ".so") != NULL) {
45              so++;
46          }
47      }
```

labs/class07-01/maps_show.c · 총 66행

32–35행 · getline 준비

NULL·0으로 시작하면 getline이 필요한 만큼 버퍼를 잡는다.

36–37행 · 반복

-1(끝 또는 오류)까지 한 행씩 세며 읽는다.

38–46행 · 분류

[heap] · [stack]은 출력하고 .so 행은 개수만 센다.

maps_show.c · 48–66행 · 오류 확인과 요약

```
48      if (ferror(fp)) {
49          perror("getline");
50          free(line);
51          fclose(fp);
52          return 1;
53      }
54      printf("regions=%ld heap_regions=%ld stack_regions=%ld"
55             " so_regions=%ld\n", total, heap, stack, so);
56      free(line);
57      if (fclose(fp) == EOF) {
58          perror("fclose");
59          return 1;
60      }
61      if (fflush(stdout) == EOF) {
62          perror("stdout");
63          return 1;
64      }
65      return 0;
66  }
```

labs/class07-01/maps_show.c · 총 66행

48–53행 · ferror

-1이 끝인지 오류인지 ferror로 구분한다.

54–55행 · 요약

전체 · heap · stack · so 구역 수를 한 줄로.

56–65행 · 정리

free, fclose 검사, 출력 flush 검사.

state_show.c · 1–11행 · 헤더와 파일 주석

```
01  #include <stdio.h>
02  #include <stdlib.h>
03  #include <string.h>
04  #include <time.h>
05  #include <unistd.h>
06
07  /* 이 프로그램은 스스로를 관찰 대상으로 만든다. 기본 모드는 1초씩
08     잠들어 ps의 STAT에서 S(sleeping)가, --busy 모드는 CPU를 놓지
09     않아 R(running/runnable)이 주로 보인다. busy 모드의 경과는
10     CLOCK_MONOTONIC으로 잰다. sleep은 중단되면 남은 시간을 돌려
11     줄 수 있지만 오늘은 1초 단위 관찰이 목적이므로 그대로 쓴다. */
```

labs/class07-01/state_show.c · 총 72행

1–5행 · 헤더

time.h는 clock_gettime,
unistd.h는 sleep·getpid.

7–9행 · 두 모드

기본은 S(잠자기),
--busy는 R(실행)을 보여 준다.

9–11행 · 시간 기준

busy 경과는 MONOTONIC으로 잰다.
sleep 중단은 오늘 무시한다.

state_show.c · 13–27행 · 단조 시계와 기본값

```
13  static double now_sec(void)
14  {
15      struct timespec ts;
16      if (clock_gettime(CLOCK_MONOTONIC, &ts) == -1) {
17          perror("clock_gettime");
18          exit(1);
19      }
20      return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
21  }
22
23  int main(int argc, char **argv)
24  {
25      int busy = 0;
26      int first = 1;
27      long seconds = 10;
```

labs/class07-01/state_show.c · 총 72행

13–21행 · now_sec

CLOCK_MONOTONIC을 초 단위 double로 돌려준다.

16–19행 · 실패

clock_gettime 실패는 복구할 수 없어 exit(1).

25–27행 · 기본값

sleep 모드, 10초, 인자 시작 위치 1.

state_show.c · 28–45행 · 모드와 초 인자 해석

```
28      if (argc > 1 && strcmp(argv[1], "--busy") == 0) {
29          busy = 1;
30          first = 2;
31      }
32      if (argc - first > 1) {
33          fprintf(stderr, "usage: %s [--busy] [seconds 1-600]\n", argv[0]);
34          return 2;
35      }
36      if (argc - first == 1) {
37          char *end = NULL;
38          seconds = strtol(argv[first], &end, 10);
39          if (end == argv[first] || *end != '\0' ||
40              seconds < 1 || seconds > 600) {
41              fprintf(stderr, "usage: %s [--busy] [seconds 1-600]\n",
42                      argv[0]);
43              return 2;
44          }
45      }
```

28–31행 · --busy

첫 인자가 --busy면 모드를 바꾸고 다음 인자부터 본다.

32–35행 · 개수

남은 인자가 둘 이상이면 사용법 오류.

36–44행 · 초

1–600 범위의 정수만 받는다.

labs/class07-01/state_show.c · 총 72행

state_show.c · 46–54행 · 관찰 안내 출력

```
46      printf("pid=%ld\n", (long)getpid());
47      printf("mode=%s seconds=%ld\n", busy ? "busy" : "sleep", seconds);
48      printf("observe: ps -o pid,ppid,stat,time,%%cpu,command -p %ld\n",
49             (long)getpid());
50      printf("observe: grep State /proc/%ld/status\n", (long)getpid());
51      if (fflush(stdout) == EOF) {
52          perror("stdout");
53          return 1;
54      }
```

46–47행 · 정체

PID와 모드·초를
먼저 알린다.

48–50행 · 명령

비교할 ps 명령과 /proc 경로를
PID를 채워 출력한다.

51–54행 · flush

관찰자가 바로 볼 수 있게
대기 전에 내보낸다.

labs/class07-01/state_show.c · 총 72행

state_show.c · 55–72행 · 1초 단위 반복

```
55      double start = now_sec();
56      for (long done = 1; done <= seconds; done++) {
57          double next = start + (double)done;
58          if (busy) {
59              while (now_sec() < next) {
60                  /* R 관찰용: CPU를 놓지 않고 기다린다 */
61              }
62          } else {
63              sleep(1);
64          }
65          printf("tick=%ld of %ld\n", done, seconds);
66          if (fflush(stdout) == EOF) {
67              perror("stdout");
68              return 1;
69          }
70      }
71      return 0;
72  }
```

labs/class07-01/state_show.c · 총 72행

55–57행 · 마감 계산

시작 시각에 done초를 더해
누적 오차 없이 마감을 정한다.

58–64행 · 두 방식

busy는 마감까지 계산 루프,
기본은 sleep(1).

65–70행 · tick 출력

매 초 진행 상황을 출력하고
flush 실패를 검사한다.

hold_open.c · 1–10행 · 헤더와 파일 주석

```
01  #include <fcntl.h>
02  #include <stdio.h>
03  #include <stdlib.h>
04  #include <unistd.h>
05
06  /* 파일을 열고 지정한 초 동안 유지하며 FD 표를 관찰하게 한다.
07     open은 남은 가장 작은 번호를 돌려주므로 표준 스트림(0·1·2)
08     만 열려 있으면 3이 돌아온다. 관찰 후 close로 반환한다.
09     실패 경로에서도 연 FD는 닫는 것이 3주차 이후의 반복 원칙이다. */
10
```

labs/class07-01/hold_open.c · 총 60행

1행 · fcntl.h

open과 O_RDONLY가 선언된 헤더다.

6–8행 · 번호 규칙

open은 가장 작은 빈 번호를 준다. 보통 3이다.

9행 · 정리 원칙

실패 경로에서도 연 FD는 닫는다.

hold_open.c · 11–31행 · 인자 해석과 open

```
11  int main(int argc, char **argv)
12  {
13      if (argc < 2 || argc > 3) {
14          fprintf(stderr, "usage: %s FILE [seconds 1-600]\n", argv[0]);
15          return 2;
16      }
17      long seconds = 10;
18      if (argc == 3) {
19          char *end = NULL;
20          seconds = strtol(argv[2], &end, 10);
21          if (end == argv[2] || *end != '\0' ||
22              seconds < 1 || seconds > 600) {
23              fprintf(stderr, "usage: %s FILE [seconds 1-600]\n", argv[0]);
24              return 2;
25          }
26      }
27      int fd = open(argv[1], O_RDONLY);
28      if (fd == -1) {
29          perror(argv[1]);
30          return 1;
31      }
```

labs/class07-01/hold_open.c · 총 60행

13–16행 · 개수

FILE은 필수, 초는 선택.
그 외는 사용법 오류.

17–26행 · 초

기본 10초, 1–600 범위만
받는다.

27–31행 · open

읽기 전용으로 열고 실패하면
파일 이름과 함께 perror.

hold_open.c · 32–49행 · 관찰 안내와 대기

```
32      printf("pid=%ld fd=%d path=%s\n", (long)getpid(), fd, argv[1]);
33      printf("hold_seconds=%ld\n", seconds);
34      printf("observe: ls -l /proc/%ld/fd\n", (long)getpid());
35      printf("observe: ls /proc/%ld/fd | wc -l\n", (long)getpid());
36      if (fflush(stdout) == EOF) {
37          perror("stdout");
38          close(fd);
39          return 1;
40      }
41      for (long done = 1; done <= seconds; done++) {
42          sleep(1);
43          printf("hold=%ld of %ld\n", done, seconds);
44          if (fflush(stdout) == EOF) {
45              perror("stdout");
46              close(fd);
47              return 1;
48          }
49      }
```

labs/class07-01/hold_open.c · 총 60행

32–35행 · 안내

PID·FD 번호와 함께 볼 /proc 명령을 출력한다.

36–40행 · flush

실패하면 FD를 닫고 종료한다.

41–49행 · 대기

1초마다 진행을 출력하며 관찰 시간 창을 연다.

hold_open.c · 50–60행 · close와 마무리

```
50      if (close(fd) == -1) {
51          perror("close");
52          return 1;
53      }
54      printf("closed fd=%d\n", fd);
55      if (fflush(stdout) == EOF) {
56          perror("stdout");
57          return 1;
58      }
59      return 0;
60  }
```

labs/class07-01/hold_open.c · 총 60행

50–53행 · close

close 실패도 보고하고 종료 상태 1.

54행 · 확인 출력

몇 번 FD를 닫았는지 남긴다.

55–59행 · flush

마지막 출력도 실패를 검사한다.

procdump.c · 1–14행 · 헤더와 파일 주석

```
001  #include <dirent.h>
002  #include <errno.h>
003  #include <stdio.h>
004  #include <stdlib.h>
005  #include <string.h>
006  #include <sys/resource.h>
007  #include <sys/types.h>
008  #include <unistd.h>
009
010  /* 프로세스의 상태와 자원을 한 보고서로 묶는다. 표준 API인
011     getrlimit·getrusage로 자원 한계와 사용량을 얻고, Linux에서는
012     /proc/self/status와 /proc/self/fd가 상태·메모리·FD 정보를
013     보충한다. /proc가 없으면 unavailable로 알린다. */
014
```

labs/class07-01/procdump.c · 총 125행

1–8행 · 헤더

dirent(디렉터리), resource(rlimit·rusage), errno.

10–11행 · 표준 API

한계와 사용량은 getrlimit · getrusage로 얻는다.

11–13행 · Linux 보충

/proc/self/status와 fd로 상태·메모리·FD를 더한다.

procdump.c · 15–31행 · show_rlimit: 한계 출력

```
015  static void show_rlimit(int resource, const char *name)
016  {
017      struct rlimit lim;
018      if (getrlimit(resource, &lim) == -1) {
019          perror("getrlimit");
020          exit(1);
021      }
022      for (int hard = 0; hard <= 1; hard++) {
023          rlim_t v = hard ? lim.rlim_max : lim.rlim_cur;
024          if (v == RLIM_INFINITY)
025              printf("%s_%s=unlimited\n", name, hard ? "hard" : "soft");
026          else
027              printf("%s_%s=%llu\n", name, hard ? "hard" : "soft",
028                     (unsigned long long)v);
029      }
030  }
031
```

labs/class07-01/procdump.c · 총 125행

17–21행 · 조회

getrlimit 실패는
exit(1)로 끝낸다.

22–23행 · soft·hard

같은 코드로 rlim_cur와
rlim_max를 차례로 다룬다.

24–28행 · 무한

RLIM_INFINITY는 숫자 대신
unlimited로 출력한다.

procdump.c · 32–48행 · 정체성과 CPU 시간

```
032  int main(int argc, char **argv)
033  {
034      if (argc != 1) {
035          fprintf(stderr, "usage: %s\n", argv[0]);
036          return 2;
037      }
038      printf("pid=%ld ppid=%ld\n",
039             (long)getpid(), (long)getppid());
040      printf("uid=%ld euid=%ld\n",
041             (long)getuid(), (long)geteuid());
042      show_rlimit(RLIMIT_NOFILE, "rlimit_nofile");
043      show_rlimit(RLIMIT_AS, "rlimit_as");
044      struct rusage ru;
045      if (getrusage(RUSAGE_SELF, &ru) == -1) {
046          perror("getrusage");
047          return 1;
048      }
```

labs/class07-01/procdump.c · 총 125행

34–37행 · 인자

인자를 받지 않는다.

38–43행 · 정체성·한계

pid·ppid, uid·euid,
NOFILE·AS 한계를 출력한다.

44–48행 · getrusage

RUSAGE_SELF로 자기 자신의
사용량을 묻는다.

procdump.c · 49–66행 · CPU 시간, maxrss, status 열기

```
049     /* suseconds_t는 시스템마다 int·long이 달라 (long)으로 맞춘다 */
050     printf("cpu_user=%ld.%06ld cpu_sys=%ld.%06ld\n",
051            (long)ru.ru_utime.tv_sec, (long)ru.ru_utime.tv_usec,
052            (long)ru.ru_stime.tv_sec, (long)ru.ru_stime.tv_usec);
053     /* 현재 RSS와 구분되는 실행 중 최대 상주 메모리다. */
054 #ifdef __APPLE__
055     printf("maxrss=%ld unit=bytes\n", ru.ru_maxrss);
056 #else
057     printf("maxrss=%ld unit=KiB\n", ru.ru_maxrss);
058 #endif
059     FILE *fp = fopen("/proc/self/status", "r");
060     if (fp == NULL) {
061 #ifdef __linux__
062         perror("/proc/self/status");
063         return 1;
064 #else
065         printf("proc_status=unavailable (no /proc)\n");
066 #endif
```

labs/class07-01/procdump.c · 총 125행

49–52행 · CPU 시간

timeval의 초·마이크로초를 %06ld로 맞춰 출력한다.

53–58행 · maxrss

최대 상주 크기. 단위가 Linux KiB, macOS 바이트로 다르다.

59–66행 · status

열기 실패 시 Linux는 오류, 다른 시스템은 unavailable.

procdump.c · 67–86행 · status에서 네 줄 고르기

```
067     } else {
068         char *line = NULL;
069         size_t cap = 0;
070         while (getline(&line, &cap, fp) != -1) {
071             if (strncmp(line, "State:", 6) == 0 ||
072                 strncmp(line, "VmSize:", 7) == 0 ||
073                 strncmp(line, "VmRSS:", 6) == 0 ||
074                 strncmp(line, "Threads:", 8) == 0) {
075                 char *colon = strchr(line, ':');
076                 *colon = '\0';
077                 char *value = colon + 1;
078                 while (*value == ' ' || *value == '\t')
079                     value++;
080                 value[strcspn(value, "\n")] = '\0';
081                 printf("status_%s=%s\n", line, value);
082             }
083         }
084         int failed = ferror(fp);
085         if (failed) perror("getline");
086         free(line);
```

labs/class07-01/procdump.c · 총 125행

68–70행 · getline

status를 한 행씩 읽는다.

71–81행 · 고르기와 다듬기

네 접두어만 골라 콜론에서
자르고 공백·줄바꿈을 지운다.

84–86행 · 오류 확인

반복이 끝나면 ferror로
읽기 오류를 확인한다.

procdump.c · 87–106행 · 정리와 FD 디렉터리 열기

```
087          if (fclose(fp) == EOF) {
088              perror("fclose");
089              failed = 1;
090          }
091          if (failed) return 1;
092      }
093      DIR *dir = opendir("/proc/self/fd");
094      if (dir == NULL) {
095  #ifdef __linux__
096          perror("/proc/self/fd");
097          return 1;
098  #else
099          printf("fd_count=unavailable (no /proc)\n");
100  #endif
101      } else {
102          /* 스캔 자체가 FD 하나를 쓰므로 Linux에서는 표준 3개보다
103             하나 더 세는 것이 정상이다. 관찰도 증거에 영향을 준다. */
104          struct dirent *ent;
105          long count = 0;
106          errno = 0;
```

87–91행 · 오류 모으기

fclose 실패도 failed에
모은 뒤 한 번에 종료한다.

93–100행 · opendir

/proc/self/fd를 연다.
없으면 unavailable.

102–106행 · 세기 준비

스캔하는 FD도 개수에 든다.
errno를 0으로 두고 시작.

labs/class07-01/procdump.c · 총 125행

procdump.c · 107–125행 · FD 세기와 마무리

```
107          while ((ent = readdir(dir)) != NULL) {
108              if (ent->d_name[0] != '.')
109                  count++;
110          }
111          int failed = errno != 0;
112          if (failed) perror("readdir");
113          printf("fd_count=%ld includes_scan_fd=yes\n", count);
114          if (closedir(dir) == -1) {
115              perror("closedir");
116              failed = 1;
117          }
118          if (failed) return 1;
119      }
120      if (fflush(stdout) == EOF) {
121          perror("stdout");
122          return 1;
123      }
124      return 0;
125  }
```

labs/class07-01/procdump.c · 총 125행

107–110행 · readdir

.으로 시작하는 항목(. ..)을 빼고 센다.

111–113행 · 결과

readdir 오류 여부와 fd_count를 출력한다.

114–124행 · 정리

closedir 검사, 출력 flush 검사 후 종료.